Cutting Page Load Time by 30% with a Tuned, Secure CDN

Cache Keys, TTLs, Purging, Origin Shielding, Compression, Request Collapsing, Health-Aware Origins and Correct Security Ordering

A simple-language guide to the project: summary, components, workflow and architecture

> **The project in one sentence (as written)**
> Cut page load time 30% for 10K+ users and shielded origins from overload by tuning CDN behavior (cache-key construction, TTLs, purge and invalidation, origin shielding, compression, request collapsing, health-aware origin selection) and sequencing normalization and security decisions correctly against cache lookup and origin forwarding so cached or transformed requests could not bypass enforcement.

Contents

1. Overall summary (in simple terms)
2. Decoding the project sentence, phrase by phrase
3. Key ideas explained simply (a mini-glossary)
4. Architecture diagram and a full walk-through of it
5. Components explained in detail
6. Workflows explained end to end
7. Where the 30% and the origin protection came from
8. Security pitfalls the correct ordering prevents
9. Quick recap

Note: the project sentence is a short summary. This guide describes a realistic, typical design for such a project so that every part of the sentence can be explained concretely. Component names, TTL values, hit ratios and timings are illustrative examples chosen for this guide, not measurements from a specific system.

1. Overall Summary (in Simple Terms)

Imagine a company with one big **central warehouse** (the **origin server**, where the website really lives) and a chain of **small local shops** spread around the country (the **CDN edge locations**, also called PoPs). Instead of every customer driving to the warehouse, they visit the shop nearest to them. If the shop already has the item on its shelf (a **cache hit**), the customer gets it immediately. If not (a **cache miss**), the shop orders it from a **regional depot** (the **origin shield**), and only if the depot does not have it either does anyone go to the central warehouse.

A **CDN (Content Delivery Network)** works exactly like this for web pages, images, scripts and some API responses. This project tuned how that CDN behaves so that:

- **Pages loaded about 30% faster** for more than 10,000 users, because far more requests were answered by the nearby shop instead of the distant warehouse, and the files were smaller.
- **The origin servers were protected from overload**, because far fewer requests reached them, and duplicate requests were merged into one.
- **Security could not be bypassed.** Every request is cleaned up (normalized) and checked by security rules **before** the shop looks on its shelf or orders from the warehouse. So a cached copy or a cleverly rewritten request could never sneak past the security guard.

The levers that were tuned, in shop language:

Lever	Shop analogy	What it achieved
Cache-key construction	The label on each shelf item. Good labels mean the same item is not stocked ten times under slightly different names.	Many more cache hits
TTLs (time to live)	Expiry dates. Long-lasting goods stay on the shelf longer; fresh goods are checked often.	Fewer unnecessary trips to the warehouse while keeping content fresh
Purge and invalidation	A product recall: remove outdated items from every shop at once.	Long TTLs became safe, because changes can be pushed out instantly
Origin shielding	A regional depot between the shops and the warehouse.	The warehouse gets one order instead of one per shop
Compression	Vacuum-packing goods so they are smaller to carry.	Smaller downloads, faster pages
Request collapsing	When 100 customers ask for the same missing item at once, the shop places one order, not 100.	No stampede on the origin when popular content expires

Lever	Shop analogy	What it achieved
Health-aware origin selection	If one warehouse is closed or struggling, order from another that is open.	Fewer errors and slow responses during problems
Correct sequencing of security	The guard checks every customer's ID before handing over anything, even items already on the shelf.	Caching never becomes a way around security

In one breath

Clean up the request → check security → build a smart label → look on the local shelf → on a miss, send one merged order to the regional depot → only then go to a healthy warehouse → vacuum-pack and shelve the answer with the right expiry date → recall it instantly when it changes. Result: 30% faster pages, calm origin servers, and no security shortcuts.

2. Decoding the Project Sentence, Phrase by Phrase

Phrase	What it means in simple words
Cut page load time 30%	The time for a page to become usable in a real user's browser dropped by about 30% (for example from 3.0 s to 2.1 s).
For 10K+ users	Measured across more than ten thousand real users, not in a lab.
Shielded origins from overload	The origin servers received far fewer requests, especially during spikes, so they stopped falling over or slowing down.
Tuning CDN behavior	Changing how the CDN decides what to cache, for how long, how to fetch from origin, and how to deliver.
Cache-key construction	Choosing exactly which parts of a request identify a cached item (host, path, some query parameters, some headers) and ignoring the rest.
TTLs	How long each kind of content may be served from cache before it must be refreshed.
Purge and invalidation	Ways to remove or mark cached items as out of date before their TTL ends.
Origin shielding	Adding a regional cache layer in front of the origin so all edge locations fetch through it.
Compression	Making text responses smaller (Brotli or gzip) before sending them.
Request collapsing	Merging many simultaneous requests for the same missing item into one origin fetch.
Health-aware origin selection	Sending origin requests only to origin servers that are currently healthy and fast.
Sequencing normalization and security decisions	Doing the steps in the right order: clean up the request first, then apply security, and only then look in the cache or forward to the origin.
Against cache lookup and origin forwarding	Relative to the two moments where a request could otherwise skip checks: being answered from cache, or being passed on to the origin.
Cached or transformed requests could not bypass enforcement	Neither a cached copy nor a rewritten or re-encoded request can avoid the security rules.

3. Key Ideas Explained Simply (a Mini-Glossary)

3.1 CDN, edge PoP and origin

- **Origin:** the company's own servers that create the real content.
- **Edge PoP (Point of Presence):** a CDN data centre close to users. There are many, in many cities.
- **Why it is faster:** data travels a short distance from a nearby PoP instead of across the world, and popular files are already waiting there.

3.2 Cache hit, miss and hit ratio

- **Hit:** the edge already has a fresh copy and answers immediately.
- **Miss:** the edge must fetch the item from further away.
- **Hit ratio:** the share of requests answered from cache. Going from 72% to 93% means misses fall from 28 in 100 to 7 in 100, so the origin sees four times less traffic.

3.3 Cache key

The cache key is the **label** used to store and find a cached response. If the key includes things that do not change the content (a tracking parameter like `utm_source`, a random cookie, the order of query parameters), the same page is stored many times under different labels and hits become rare. If the key leaves out something that does change the content (for example the language or the logged-in user), different users could get the wrong page. A good key includes exactly what matters and nothing more.

3.4 TTLs and freshness

- **TTL:** how long a cached copy is considered fresh.
- **Cache-Control headers:** instructions from the origin, such as `max-age` (browser), `s-maxage` (shared caches like a CDN), `private` (do not store in shared caches) and `no-store` (never store).
- **stale-while-revalidate:** after the TTL ends, briefly keep serving the old copy while fetching a new one in the background, so users never wait.
- **stale-if-error:** if the origin is failing, keep serving the old copy for a while instead of showing an error.
- **Revalidation:** asking the origin "has this changed?" using `ETag` or `Last-Modified`. If not, the origin answers "304 Not Modified" without sending the whole file.

3.5 Purge and invalidation

- **Hard purge:** delete the item from cache now; the next request fetches a new copy.
- **Soft purge (invalidation):** mark the item stale; it can still be served briefly while a fresh copy is fetched.
- **Surrogate keys / cache tags:** labels attached to responses (for example `product-123`) so one purge call can remove every page that shows that product.

- **Versioned file names:** static files named with a content hash (`app.3f9a1c.js`) never need purging; a new version simply has a new name.

3.6 Origin shield

A shield is a designated regional cache that sits between all edge PoPs and the origin. Without it, 50 PoPs missing the same item send 50 requests to the origin. With it, they all ask the shield, and the shield asks the origin once.

3.7 Compression

Text files (HTML, CSS, JavaScript, JSON, SVG) compress very well. **Brotli** usually makes them 15–25% smaller than **gzip**. The browser says which it supports in the `Accept-Encoding` header, and the CDN keeps separate cached variants for each encoding. Images and videos are already compressed and are not compressed again.

3.8 Request collapsing

Also called request coalescing. When many requests for the same uncached item arrive at the same moment, only the first goes upstream; the rest wait a few milliseconds and receive the same answer. This prevents a **thundering herd** (a stampede of identical requests) when a popular item expires or is purged.

3.9 Health-aware origin selection

The CDN regularly checks each origin (health probes) and watches real responses (error rates, timeouts, slowness). It then sends traffic only to healthy origins, preferring the closest and least loaded, and fails over to a backup region when needed.

3.10 Normalization and why order matters

Normalization rewrites a request into one standard (canonical) form: lowercase host name, decode percent-encoding exactly once, remove `./` and `../` path tricks, merge double slashes, sort query parameters, drop tracking parameters and unexpected headers. If security rules look at one version of the request and the cache or origin uses another, an attacker can hide a forbidden request inside a disguise that only one side recognises. Normalizing **first** means every later step sees the same request.

3.11 Page load metrics

- **TTFB (Time to First Byte):** how quickly the first byte of a response arrives. Caching at the edge improves it most.
- **LCP (Largest Contentful Paint):** when the main content appears. Compression and fast static files improve it.
- **RUM (Real User Monitoring):** measurements collected from real users' browsers, which is how the 30% for 10K+ users is measured.

4. Architecture Diagram and Full Walk-Through

The diagram on the next page shows how a request flows through an edge location, the control plane that configures it, the origin shield and the origin servers. Numbered circles show the order of events.

How to read the diagram	Meaning
Blue zone (left)	The edge PoP near users. Boxes 1 to 7 are the stages every request passes through, in order.
Red box (3)	Security enforcement, highlighted because its position in the order is the key security design decision.
Orange zone (right), purple boxes	The control plane: observability, health checks, security rules, cache rules and purge.
Orange zone (right), orange boxes	The origin shield and the health-aware origin selector.
Green boxes	Users, operators, the delivered response, and the origin servers.
Dark blue arrows 1–11	The request path for a cache miss, from user to origin and back.
Green dashed arrow H	The cache hit shortcut: the response comes straight from the edge cache.
Red arrow S	Security rules pushed to the enforcement stage.
Purple arrows K, P1, P2, HC, M	Cache-key rules, purges to edge and shield, origin health status, and metrics.

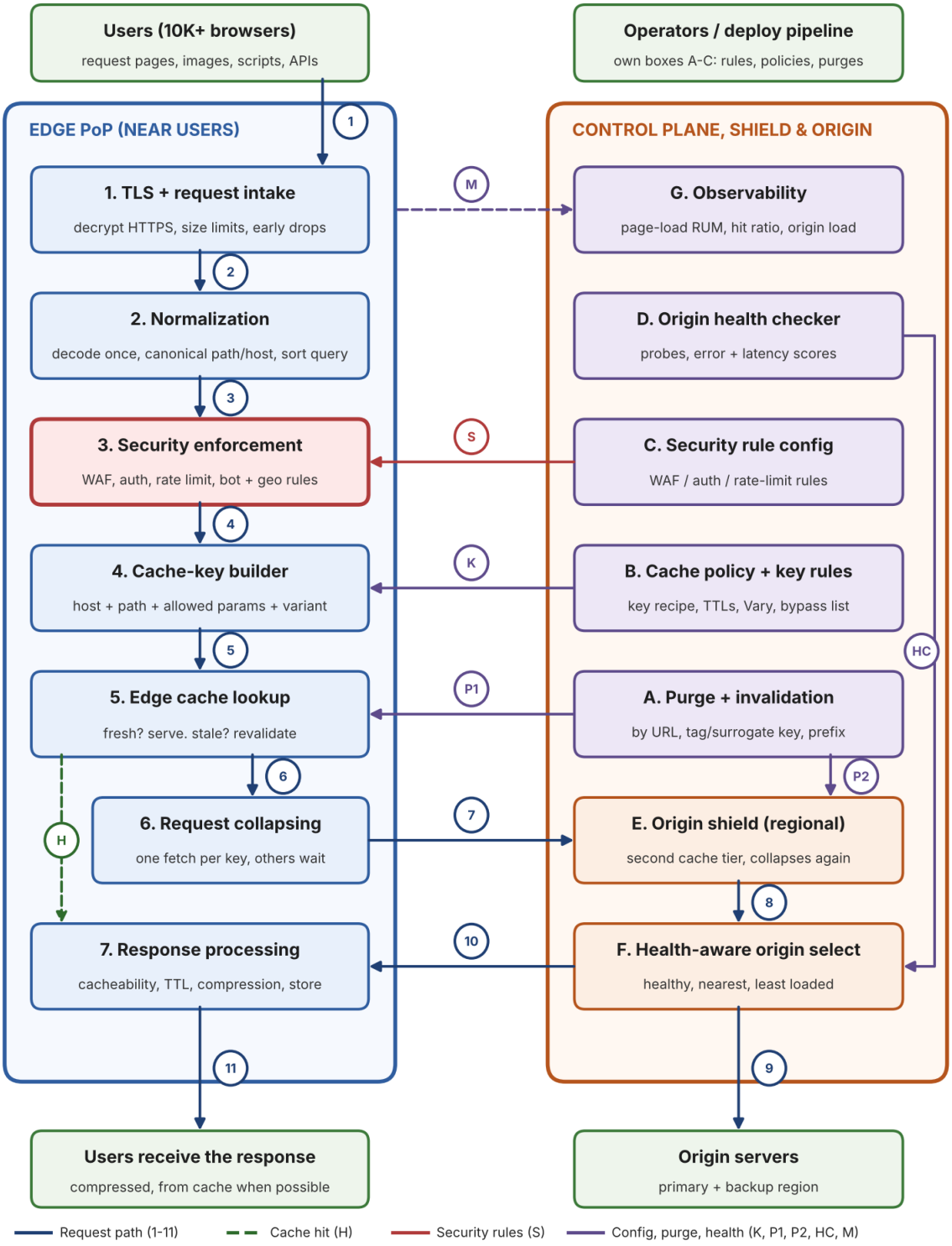


Figure 1. Tuned CDN: edge pipeline with security before cache, control plane, origin shield and health-aware origin selection

4.1 The zones

Blue zone – edge PoP. This is where users connect. The order of boxes is deliberate: intake, then normalization, then security, and only after that the cache key and cache lookup. Response processing sits at the bottom, where answers from the origin are made cacheable, compressed and stored.

Orange zone – control plane, shield and origin side. The purple boxes do not handle user traffic directly; they configure the edge (rules, keys, purges), watch origin health, and collect measurements. The orange boxes are the second cache tier (shield) and the component that chooses which origin to call.

Green boxes – outside world. Users at the top and bottom of the left side (request in, response out), operators who own the rules, and the origin servers at the bottom right.

4.2 Every box in the diagram

Box	Zone	One-line explanation
Users (10K+ browsers)	External	Real people loading pages, images, scripts and API data.
Operators / deploy pipeline	External	Engineers and automated deploys that own the rules (C, B) and trigger purges (A).
1. TLS + request intake	Edge	Decrypts HTTPS, enforces size limits, drops obviously broken requests early.
2. Normalization	Edge	Turns the request into one canonical form so every later step sees the same thing.
3. Security enforcement	Edge (red)	WAF, authentication and authorization, rate limits, bot and geo rules. Runs before any cache answer.
4. Cache-key builder	Edge	Builds the label from host, path, allowed query parameters and real variants.
5. Edge cache lookup	Edge	Finds a fresh copy (hit), a stale copy to revalidate, or nothing (miss).
6. Request collapsing	Edge	Merges simultaneous misses for the same key into one upstream fetch.
7. Response processing	Edge	Decides cacheability and TTL, compresses, stores, and sends to the user.
A. Purge + invalidation	Control plane	Removes or marks stale cached items by URL, tag or prefix, at edge and shield.

Box	Zone	One-line explanation
B. Cache policy + key rules	Control plane	The key recipe, TTLs per content type, Vary rules and the list of paths never cached.
C. Security rule config	Control plane	WAF rules, auth requirements, rate limits, bot and geo policies.
D. Origin health checker	Control plane	Probes origins and scores them on errors and latency.
E. Origin shield (regional)	Shield	Second cache tier; all edges fetch through it; collapses requests again.
F. Health-aware origin select	Shield	Picks a healthy, near, lightly loaded origin; fails over when needed.
G. Observability	Control plane	Real-user page load times, hit ratio, origin load and error dashboards.
Users receive the response	External	The compressed response arrives, from cache whenever possible.
Origin servers	External	The company's real servers, in a primary and a backup region.

4.3 Every arrow in the diagram

Arrow	From → To	What travels along it
1	Users → Intake	An HTTPS request for a page or file.
2	Intake → Normalization	The decrypted raw request.
3	Normalization → Security	The canonical request (one agreed form).
4	Security → Cache-key builder	Only requests that passed security, plus any security context needed for the key (for example the user's access tier).
5	Key builder → Cache lookup	The cache key.
H	Cache lookup → Response processing	Cache hit: the stored response goes straight out (green dashed shortcut).
6	Cache lookup → Collapsing	Cache miss (or expired item needing a refresh).
7	Collapsing → Origin shield	One request per key travels to the regional shield.

Arrow	From → To	What travels along it
8	Shield → Origin selector	Only if the shield also misses.
9	Origin selector → Origin servers	The request goes to the chosen healthy origin, with only allowed headers.
10	Origin selector → Response processing	The origin response comes back through the shield (which stores it) to the edge.
11	Response processing → Users	The final, compressed response, now also stored at the edge for the next user.
S	Security rule config → Security	Rule updates pushed to every edge.
K	Cache policy → Key builder	Key recipe, TTLs and bypass list pushed to every edge.
P1	Purge → Edge cache	Purge or invalidate items in every edge cache.
P2	Purge → Shield	Purge the same items in the shield, so edges do not refill from an old shield copy.
HC	Health checker → Origin selector	Live health scores for each origin.
M	Edge → Observability	Logs and metrics from every stage (dashed: telemetry, not user traffic).

4.4 Why the diagram is shaped this way

- **Normalization and security sit above the cache.** Nothing can be served, from cache or from origin, until the request has been cleaned up and checked. This is the heart of "could not bypass enforcement".
- **The hit path (H) skips only work, never security.** It starts below the security box, so even a cached answer has already passed every check.
- **Two cache tiers and two collapsing points (6 and E).** Misses are filtered twice before any request reaches the origin.
- **Purge reaches both tiers (P1 and P2).** Otherwise an edge could refill itself from an old shield copy.
- **Origin selection is fed by live health (HC).** Origin traffic avoids servers that are failing or slow.

5. Components Explained in Detail

Each component below has a fact table followed by a step-by-step description of what happens inside it.

5.1 External systems

Users (10K+ browsers)

What it is	Real people using the website or app from many locations, devices and networks.
Simple analogy	Customers walking into their nearest shop.
What they request	HTML pages, CSS, JavaScript, images, fonts, and some API responses.

How it works, step by step

1. The browser looks up the site's address and is directed to the nearest CDN edge PoP.
2. It opens an HTTPS connection and sends requests, saying which compression it supports (`Accept-Encoding`).
3. A RUM script on the page reports how long the page took to load (feeding G).

Operators / deploy pipeline

What it is	The engineers and automated release process that own CDN configuration.
Simple analogy	Head office staff who write the shop rules and issue recalls.
Responsibilities	Review and deploy security rules (C) and cache rules (B); trigger purges (A) when content changes.

How it works, step by step

1. Changes to rules are stored as code, reviewed, and tested in staging.
2. Deploys roll out gradually to edges, watching G for problems.
3. When content is published or changed, the deploy pipeline calls the purge API with the relevant tags.

5.2 Edge PoP components

1. TLS + request intake

What it is	The edge's front door: TLS decryption and basic request acceptance.
Simple analogy	The shop entrance.
Key details	Modern TLS with session resumption; HTTP/2 and HTTP/3 to users; strict limits on header size and count.

How it works, step by step

1. Terminates TLS and decrypts the request.
2. Rejects requests that are clearly malformed or exceed size limits.
3. Records the arrival time and basic details for metrics (arrow M).
4. Passes the raw request to normalization (arrow 2).

2. Normalization

What it is	A set of rules that rewrites every request into one canonical form.
Simple analogy	Writing every customer's order neatly on a standard form before anyone reads it.
Why first	Security (3), the cache key (4) and the origin request (9) must all be based on the same request. If they each interpreted the raw request differently, an attacker could disguise a request so that security sees something harmless while the cache or origin sees something dangerous.
Typical rules	Lowercase host; remove default ports; decode percent-encoding exactly once and reject double-encoding; resolve `./` and `../`; collapse `//`; reject ambiguous characters; sort query parameters; drop tracking parameters (`utm_*`, `fbclid`); remove or overwrite untrusted headers such as `X-Forwarded-Host`.

How it works, step by step

1. Parses the URL and headers.
2. Applies each normalization rule in a fixed order.
3. Rejects requests that cannot be normalized safely (ambiguous encodings, illegal characters) with a 400 error.
4. Strips headers the origin should never receive from users, and sets trusted ones itself (for example the real client IP).
5. Hands the canonical request to security (arrow 3).

3. Security enforcement

What it is	Every security decision made at the edge: WAF (attack patterns), authentication and authorization, rate limiting, bot detection, geo restrictions.
Simple analogy	The guard who checks every customer's ID and bag before they reach any shelf.
Position	After normalization, before the cache key, cache lookup and origin forwarding. This is the core sequencing decision.
Why before the cache	If the cache were checked first, a response stored from an authorised user could be served to an unauthorised one, and blocked requests could still be answered from cache. Checking first makes caching a pure speed-up, never a shortcut around rules.

How it works, step by step

1. Receives the canonical request.
2. Applies WAF rules (SQL injection, path traversal, known exploits) to the canonical form, so encoding tricks cannot hide attacks.
3. Checks rate limits per client, token and route.
4. For protected content, validates the user's token or session at the edge on **every** request, including ones that will be cache hits.
5. Determines an access context (for example "anonymous", "premium") that the key builder may include, so different access levels never share a cached copy.
6. Blocks (403/429) or passes the request to the key builder (arrow 4).

4. Cache-key builder

What it is	Builds the label used to store and find responses, following the key recipe from B (arrow K).
Simple analogy	A precise product label: same product, same label, always.
Problem before	Keys included the full query string (including tracking parameters and parameter order) and some cookies, so one page existed as hundreds of separate cache entries and the hit ratio was low.
Recipe after	Scheme + normalized host + normalized path + an allowlist of query parameters that actually change the response (sorted) + real variants only (compression encoding, device class or language when the page truly differs) + access context for protected content. Cookies and other headers are excluded unless explicitly needed.

How it works, step by step

1. Reads the normalized host and path.
2. Keeps only allowlisted query parameters, sorted alphabetically; drops the rest.
3. Adds variant dimensions defined in B (for example `br`, `gzip` or none).
4. Adds the access context from security for protected routes, or marks the request as **bypass** (never cached) for personal pages such as account or checkout.
5. Produces the key and passes it to the cache lookup (arrow 5).

5. Edge cache lookup

What it is	The local cache at this PoP (memory and fast disk).
Simple analogy	Checking the shop shelf.
Possible outcomes	Fresh hit; stale hit (serve and refresh in background); stale but origin failing (serve under stale-if-error); miss; bypass.

How it works, step by step

1. Looks up the key.
2. **Fresh hit:** sends the stored response straight to response delivery (arrow H).
3. **Stale, within stale-while-revalidate:** serves the stale copy immediately and triggers a background refresh through collapsing.
4. **Miss or bypass:** passes the request to collapsing (arrow 6).
5. Applies purge and invalidation messages from A (arrow P1) as they arrive.

6. Request collapsing

What it is	A waiting room for identical misses.
Simple analogy	One restock order for 100 customers asking for the same missing item.
Key details	Only cacheable requests are collapsed (never personal or bypass requests); waiting time is capped; if the first fetch returns an uncacheable response, waiting requests are released to fetch on their own.

How it works, step by step

1. Checks whether a fetch for this key is already in progress.
2. If yes: the request waits for that fetch's result.
3. If no: marks the key as in progress and sends one request to the shield (arrow 7).
4. When the response arrives, all waiting requests receive it at once.

7. Response processing

What it is	Everything done to a response before it is stored and delivered.
Simple analogy	Checking the delivery, vacuum-packing it, putting an expiry date on it, shelving a copy and handing one to the customer.
Cacheability rules	Never cache responses with `Set-Cookie`, `Cache-Control: private` or `no-store`; never cache personal pages; decide by the origin's headers and content type, not by file extension (prevents cache deception); cache errors only very briefly.
Compression	Brotli for clients that support it, gzip otherwise, for text types above a minimum size; skip already-compressed formats; add `Vary: Accept-Encoding`.

How it works, step by step

1. Receives the origin response (arrow 10) or the cached response (arrow H).
2. For origin responses, decides whether it may be cached and for how long, using the origin's headers and the TTL rules in B.
3. Compresses text responses if the origin did not already, and stores a compressed variant per encoding.
4. Stores cacheable responses in the edge cache under the key.
5. Sends the response to the user (arrow 11) and records timing and cache status (HIT, MISS, STALE) for metrics.

5.3 Control plane, shield and origin components

A. Purge + invalidation

What it is	An authenticated API and distribution system that removes or invalidates cached items across every edge and the shield.
Simple analogy	A product recall sent to every shop and the depot at the same time.
Methods	By exact URL; by surrogate key / tag (all pages showing product 123); by prefix (everything under `/blog/`); soft purge (mark stale) or hard purge (delete).
Why it enabled faster pages	Because changes can be removed instantly, TTLs could safely be made much longer, which raised the hit ratio.

How it works, step by step

1. The deploy pipeline or CMS calls the purge API with tags or URLs; the API checks the caller is authorised.
2. The purge is sent to the shield (arrow P2) and every edge (arrow P1).
3. Soft purges mark items stale so the next request refreshes them while users still get a fast answer.
4. Completion is confirmed and logged; purge rates are limited so a mistake or abuse cannot empty all caches at once.

B. Cache policy + key rules

What it is	Version-controlled configuration that defines keys, TTLs and caching behaviour per route and content type.
Simple analogy	The shop manual: how to label items, and the expiry date for each kind of product.
Example TTLs	Versioned static files: 1 year (immutable). Images: 1 day to 1 week. HTML pages: 1–5 minutes with stale-while-revalidate. Public API data: a few seconds. Personal pages: never cached.

How it works, step by step

1. Defines the key recipe (arrow K): allowed query parameters, variants, bypass routes.
2. Defines TTLs and stale settings, respecting the origin's `Cache-Control` where present.
3. Is reviewed and tested, then rolled out to edges gradually.
4. Is tuned using hit-ratio data from G.

C. Security rule config

What it is	The definitions used by security enforcement (3).
Simple analogy	The guard's instructions.
Contents	WAF rule sets, routes requiring authentication, rate limits, bot policies, allowed countries.

How it works, step by step

1. Rules are written and tested against normalized example requests.
2. They are pushed to every edge (arrow S) and take effect for all requests, cached or not.
3. Blocks and false positives are monitored in G and rules are adjusted.

D. Origin health checker

What it is	A service that continually measures the health of each origin server or region.
Simple analogy	Phoning each warehouse regularly to ask if it is open and how busy it is.
Signals	Active probes to a health URL; passive signals from real traffic (error rate, timeouts, latency).

How it works, step by step

1. Sends probes every few seconds to each origin.
2. Combines probe results with real traffic statistics into a health score.
3. Marks an origin unhealthy after several failures and healthy again after several successes (to avoid flapping).
4. Sends scores to the origin selector (arrow HC).

E. Origin shield (regional)

What it is	A designated cache location that all edges use when they miss.
Simple analogy	The regional depot between shops and the warehouse.
Effect	Turns many edge misses into one origin request, and collapses simultaneous requests a second time.

How it works, step by step

1. Receives collapsed misses from many edges (arrow 7).
2. Checks its own cache: on a hit, returns the response to the edge.
3. On a miss, collapses identical requests and passes one to the origin selector (arrow 8).

4. Stores the origin's response (respecting cacheability rules) and returns it to all waiting edges.
5. Applies purges (arrow P2).

F. Health-aware origin select

What it is	The logic that chooses which origin server or region receives a request.
Simple analogy	Ordering from whichever warehouse is open, nearby and not swamped.
Rules	Use only healthy origins; prefer the nearest; balance by load; limit concurrent connections per origin; short timeouts; retry once on another origin for safe (idempotent) requests; fall back to stale content if all origins fail.

How it works, step by step

1. Reads current health scores (arrow HC).
2. Chooses an origin and forwards the request with only allowed, trusted headers (arrow 9).
3. If the origin fails or times out, retries a different healthy origin for safe requests.
4. If every origin is unhealthy, signals the caches to serve stale content (stale-if-error) instead of errors.
5. Returns the response through the shield to the edge (arrow 10).

G. Observability

What it is	Dashboards and alerts built from CDN logs and real-user measurements.
Simple analogy	Sales and stock reports for every shop.
Key metrics	Page load time (RUM: TTFB, LCP) per region; hit ratio at edge and shield; origin requests per second; bandwidth saved; compression ratio; error and block rates.

How it works, step by step

1. Collects logs and metrics from edges (arrow M) and RUM beacons from browsers.
2. Shows before/after comparisons for each tuning change.
3. Alerts on falling hit ratio, rising origin load, or rising errors.

Users receive the response and origin servers

Users receive the response	The final step of the request path: a compressed response, usually from the nearby edge cache.
Origin servers	The company's application servers in a primary and a backup region. After the project they handled a small fraction of the previous traffic and stayed stable during spikes.

How it works, step by step

1. The browser receives and decompresses the response, then renders the page.
2. The origin sends correct `Cache-Control`, `ETag` and surrogate-key headers so the CDN can cache safely.
3. The origin answers revalidation requests with 304 Not Modified when nothing changed.

6. Workflows Explained End to End

6.1 The sequencing rule (the security workflow)

This is the order every request follows at the edge, and why each step must come where it does.

Order	Step	Why here	What goes wrong if out of order
1	Intake	Accept and decrypt.	–
2	Normalize	Create one canonical request for all later steps.	Security, cache and origin read different versions; disguised attacks slip through.
3	Security	Decide allow/deny on the canonical request.	If after cache lookup: cached content is served to blocked or unauthorised users.
4	Build cache key	Uses only normalized, allowed parts plus access context.	If built from the raw request: duplicate entries, poisoning, or users sharing private copies.
5	Cache lookup	Serve allowed requests fast.	If before security: caching becomes a bypass.
6	Forward to origin	Only allowed, normalized requests with clean headers.	Origin could receive attack payloads or forged headers.
7	Response policy	Store only safe, public responses.	Personal data could be cached and shown to others.

6.2 Cache hit workflow (the fast path)

1. The user requests a page; the nearest edge receives it (arrow 1).
2. The request is normalized (arrow 2 to 3) and checked by security (arrow 3 to 4).
3. The cache key is built (arrow 5).
4. The edge finds a fresh copy and sends it straight out (arrow H), already compressed for the user's browser.
5. The response is logged as a HIT. Total time: typically tens of milliseconds, with no origin involvement.

6.3 Cache miss workflow (the full path)

1. Steps 1–5 run as above, but the edge has no fresh copy.

2. **Collapsing (arrow 6):** if another request for the same key is already fetching, wait for it; otherwise become the one fetcher.
3. **Shield (arrow 7):** the fetch goes to the regional shield. If the shield has it, it returns immediately.
4. **Shield miss (arrow 8):** the shield collapses again and asks the origin selector.
5. **Origin selection (arrow 9):** a healthy, nearby origin is chosen and receives the clean, normalized request.
6. **Response (arrow 10):** the origin replies; the shield stores a copy; the edge receives it.
7. **Response processing:** cacheability and TTL decided, compressed if needed, stored at the edge.
8. **Delivery (arrow 11):** the user and every waiting collapsed request receive the response.

6.4 Revalidation and stale-while-revalidate workflow

1. A cached HTML page reaches the end of its TTL (for example 5 minutes).
2. The next request is still served instantly from the stale copy (within the stale-while-revalidate window).
3. In the background, one collapsed request asks the origin “has it changed?” using `If-None-Match` with the stored ETag.
4. If unchanged, the origin replies 304 (no body); the edge simply resets the TTL.
5. If changed, the origin sends the new page; it is stored and served to the next users.
6. Users never wait for the refresh, which removes slow requests from page loads.

6.5 Purge and invalidation workflow

1. An editor updates product 123 in the CMS, or a new release is deployed.
2. The CMS or pipeline calls the purge API with the tag `product-123` (pages) or relies on new hashed file names (static assets).
3. The API authenticates the caller and checks rate limits.
4. The purge is applied at the shield first (P2), then at every edge (P1), so edges cannot refill from an old shield copy.
5. For pages, a soft purge marks them stale: the next request gets a quick response while a fresh copy is fetched.
6. The purge completes worldwide within seconds and is logged.

6.6 Health-aware origin selection and failover workflow

1. The health checker probes each origin every few seconds and watches real error and latency rates.
2. Normal state: requests go to the nearest healthy origin, balanced by load.
3. An origin starts failing: after several failures it is marked unhealthy and removed from selection.
4. Traffic moves to the next healthy origin (for example the backup region).
5. If all origins fail, caches serve stale content (stale-if-error) instead of errors.

6. When the failed origin passes several probes in a row, it is gradually returned to service.

6.7 Compression workflow

1. The browser sends `Accept-Encoding: br, gzip`.
2. The cache key includes the chosen encoding, so Brotli and gzip versions are stored separately.
3. On a miss, the edge asks the origin for an uncompressed (or Brotli) version.
4. Response processing compresses text types above a minimum size (for example 1 KB) with Brotli for supporting browsers and gzip for others; images and video are left alone.
5. The compressed variant is stored, so compression work happens once per variant, not once per user.

6.8 Cache-key tuning workflow

1. Analyse CDN logs: for each path, how many different keys exist and what makes them different.
2. Find parameters and headers that do not change the response (tracking parameters, parameter order, random cookies).
3. Build an allowlist of parameters that do matter per route; remove everything else from the key.
4. Find routes that truly vary by language or device and add only those variants.
5. Mark personal routes (account, checkout, cart APIs) as bypass so they are never cached.
6. Test in staging that different users never receive each other's content, then roll out gradually and watch the hit ratio rise.

6.9 TTL tuning workflow

Content type	Example TTL	Reasoning
Versioned static files (app.3f9a1c.js)	1 year, immutable	The name changes when content changes, so the old file never needs updating.
Images and fonts	1 day – 1 week	Rarely change; can be purged by tag if needed.
HTML pages	1–5 minutes + stale-while-revalidate	Change more often; purge on publish keeps them current.
Public API responses	5–60 seconds	Short TTL still absorbs huge bursts of identical requests.
Error responses (404, 5xx)	A few seconds	Prevents hammering the origin, without keeping errors around.
Personal or logged-in content	Not cached (bypass)	Must never be shared between users.

6.10 Rollout and measurement workflow

1. Record a baseline: RUM page load times, hit ratio, origin requests per second.
2. Apply one change at a time (for example the new key recipe) to a small percentage of traffic or a few PoPs.
3. Compare with the rest of traffic; check correctness (no wrong content) and security block rates.
4. Expand gradually; roll back instantly if problems appear.
5. After all changes, compare full weeks before and after: page load about 30% faster across 10K+ users; origin traffic sharply lower.

7. Where the 30% and the Origin Protection Came From

The figures below are **illustrative**, showing how the levers combine.

Metric	Before	After	Main cause
Edge hit ratio	72%	93%	Cleaner cache keys, longer TTLs made safe by purging, stale-while-revalidate
Average text transfer size	100%	about 75%	Brotli compression stored per variant
Origin requests per second at peak	100%	about 15%	Higher hit ratio, origin shield, request collapsing
Origin error spikes during traffic bursts	Frequent	Rare	Collapsing, shield, health-aware selection, stale-if-error
Median page load time (RUM)	3.0 s	2.1 s	All of the above: faster first byte and smaller files

Example funnel for 10,000 requests to a popular page at peak (after tuning):

Stage	Requests remaining	What happened
Arrive at edges	10,000	All users' requests.
Blocked by security	9,900	About 100 malicious or over-limit requests stopped before any cache or origin work.
Served from edge cache	9,200 hits	Only 700 misses continue.
After edge request collapsing	300	Simultaneous identical misses merged.
Served by shield	250 hits	Only 50 continue.
Reach the origin	about 50	The origin sees 0.5% of the original traffic.

8. Security Pitfalls the Correct Ordering Prevents

Attack or mistake	How it works (simply)	How this design prevents it
Auth bypass via cache	A page cached for a logged-in user is served to someone who is not logged in.	Security runs before lookup on every request; protected content is bypassed or keyed by access context.
Cache poisoning	An attacker sends a request with an unkeyed header (for example `X-Forwarded-Host: evil.com`) that changes the response; the bad response is cached for everyone.	Normalization strips or overwrites untrusted headers before the origin sees them; keys and forwarded headers are allowlisted.
Web cache deception	An attacker tricks a victim into visiting `/account/profile.css`; the CDN caches the personal page because it "looks like" CSS.	Cacheability is decided by origin headers and content type, never by file extension; personal routes are bypassed.
Path confusion / encoding tricks	`/admin%2f..` or double-encoded paths look harmless to one component and dangerous to another.	Decode once, canonicalize and reject ambiguity before security and before building the key.
Blocked request answered from cache	A WAF rule blocks a pattern, but a cached response for that URL is served anyway.	Security decides first; cached answers only go to requests that passed.
Purge abuse	Someone floods the purge API to empty caches and overload the origin.	Purge API requires authentication, is rate-limited and logged.
Origin bypassing the CDN	Attackers hit the origin directly, skipping all edge security.	Origin accepts traffic only from the CDN (allowlisted IPs or a secret header / mutual TLS).

9. Quick Recap

Question	Short answer
What was achieved?	About 30% faster page loads for 10K+ users and far less load on origin servers.
How was caching improved?	Clean, allowlisted cache keys; TTLs per content type; stale-while-revalidate; purge by tag so long TTLs stay safe.
How was the origin protected?	Origin shield, request collapsing at edge and shield, health-aware origin selection, stale-if-error.
How were files made smaller?	Brotli/gzip compression of text, stored once per encoding variant.
What was the security insight?	Normalize first, enforce security second, and only then look in the cache or forward to the origin.
Why does that order matter?	So that neither cached copies nor rewritten requests can ever skip the security rules.